

Project OS — Référence complète des fonctionnalités

v0.1.0 Extension VS Code / Antigravity + CLI C++ + moteur IA local (LocalAI). **Toutes** les capacités, commandes, modules, tests et artefacts — sans omission.

Ce document couvre : les 8 commandes slash, ~23 commandes VSCode, 13 modules projets, le routing IA, le budget, les tokens, les artefacts, l'autonomie, l'endurance GPU, les guards de sécurité, la config, le CLI C++, les tests (247+) et les artefacts livrés.

Sommaire

- 1. Architecture & périmètre
- 2. Commandes slash (/)
- 3. Commandes VS Code (`clineProjectOS.*`)
- 4. Modules « projets » (13)
- 5. Routing intelligent des modèles
- 6. Budget & coût
- 7. Télémétrie tokens & coût
- 8. Artifacts (artefacts)
- 9. Autonomie (write-lane gated)
- 10. Endurance GPU Ladder
- 11. Guards de sécurité
- 12. Télémétrie / observabilité / dashboard
- 13. CLI C++ (menus)
- 14. Configuration (settings)
- 15. Tests & baseline
- 16. Artefacts livrés (phases 13-23)
- 17. Chemins & prérequis

1. Architecture & périmètre

Project OS = **hub-spoke** : un « control plane » (l'extension) gère des *workspaces de projets enfant* créés par `/create` . Chaque projet enfant est auto-gouverné (goal, todo, addons, autonomy, artifacts).

Couche	Rôle	Emplacement
Extension VS Code	Commandes, arbres, statut bar, config	<code>dist/extension.js</code>
Domain (pur)	Logique métier testable (aucun vscode)	<code>src/*</code>
Moteur IA	Génération via LocalAI	<code>http://127.0.0.1:8080/v1</code>
Projets enfant	Workspaces gérés	<code>C:\Users\eiden\Desktop\dev\projects\<slug></code>
CLI C++	Pilotage par menus	<code>cli-cpp\</code>

2. Commandes slash (/)

Utilisables dans la console d'input (`clineProjectOS.slash`) et par le CLI C++. Préfixe `/`, args + `--flags` (respect des guillemets).

Commande	Sous-commandes	Description
<code>/create</code>	<code>--type=<t></code> <code>--git</code> <code>--objective</code>	Crée un projet géré (hub-spoke) + seed le TODO. Types: cpp, ts, web, node, rust, go, python, desktop, docker, localai, auto.
<code>/goal</code>	<code><objective></code> <code>--criteria=<c></code>	Définit/maj l'objectif + critères mesurables du projet actif (<code>goal.json</code> + historique).
<code>/todo</code>	<code>list</code> <code>add <label></code> <code>done <key></code>	Liste (items barrés si faits), ajoute, marque fait. Persisté (todo.json + TODO.md).
<code>/addon</code>	<code>list</code> <code>recommended</code> <code>add <id></code> <code>remove</code> <code>disable</code> <code>enable</code>	Gère les addons workspace (core, cpp, web, node, python...).
<code>/autonomy</code>	<code>plan --minutes=<n></code> <code>run</code> <code>summary</code> <code>--write</code>	Plan d'autonomie (budget minutes par complexité), exécution agent, résumé. <code>--write</code> = write-lane borné + gated.
<code>/docs</code>	<code><domain></code> <code>--category=<c></code>	Navigation des sources officielles en ligne pour un domaine (SFL, C++, Win32...).
<code>/project</code>	<code><slug></code> <code>--project=<slug></code>	Résumé d'un projet géré (type, statut, goal, progress).
<code>/report</code>	<code>--job=<id></code> <code>--model=<id></code>	Rapport tokens + coût + perf d'un job (vrai total, jamais inventé).

3. Commandes VS Code (`clineProjectOS.*`)

Enregistrées dans `contributes.commands` + `views` + arbres.

Commande	Description
<code>controlcenter.open</code>	Ouvre le Control Center (dashboard des artifacts).
<code>artifact.create</code>	Crée un artifact (choix type + titre + contenu).
<code>artifact.open</code> / <code>artifact.search</code>	Ouvre / recherche un artifact.
<code>agent.start</code> / <code>agent.abort</code>	Démarre / interrompt un agent (session Cline).
<code>provider.preflight</code>	Préflight fournisseur (compatibilité, streaming, santé).
<code>project.scan</code>	Analyse l'ADN du projet (ProjectDNA).
<code>workspace.report</code>	Rapport workspace (topologie single/multi/empty).
<code>tokens.open</code> / <code>tokens.refresh</code> / <code>tokens.export</code>	Ouvre la synthèse tokens, rafraîchit, exporte le rapport.
<code>mode.route</code>	Route « pourquoi ce modèle ? » pour une classe de tâche.
<code>budget.status</code>	Statut budget PAOG (daily/actual/reserved/remaining).
<code>antigravity.verifyWorkspace</code>	Valide le workspace + gate de redémarrage IDE (jamais Windows).
<code>goal</code> / <code>create</code> / <code>addon</code> / <code>autonomy</code> / <code>slash</code>	Équivalents des slash-commandes (invite vs input).

<code>projects.refresh</code> / <code>project.status</code>	Rafraîchit l'arbre / affiche le statut d'un projet.
---	---

Vues : `cLineProjectOS.artifacts` (arbre artifacts), `cLineProjectOS.projects` (arbre projets), `ControlCenterProvider`.

4. Modules « projets » (13)

Module	Fonction
<code>ProjectFactory</code>	Création transactionnelle de projet enfant + slugify + anti path-traversal.
<code>ManagedProjectRegistry</code>	Registre hub (persisté JSON) : list/get/add/update/ remove .
<code>GoalService</code>	Objectif + critères + progress + historique.
<code>TodoTracker</code> / <code>FsTodoIO</code>	TODO durable (todo.json + TODO.md), items barrés.
<code>AddonManager</code> / <code>AddonCatalog</code>	Addons workspace install/list/recommended/disable/enable.
<code>AutonomyService</code> / <code>AutonomyRunner</code> / <code>AutonomyModelSelector</code>	Plan autonome, exécution agent, sélection de modèle.
<code>AutonomyWriteScope</code> / <code>AutonomyWriteLane</code>	Write-lane borné : plan + scope + guard de chemins (protected/secret).
<code>AutonomyActivityLedger</code>	Journal d'activité chronométré (budget minutes).
<code>SlashCommands</code> (Registry + handlers)	Parsing slash + dispatch + handlers /goal /create /addon /autonomy /todo.
<code>ProductCommands</code>	Handlers /docs /project /report.
<code>ProjectDNA</code>	ADN projet (scan workspace).
<code>DocsNavigator</code> / <code>DocsCodingAdviser</code> / <code>DocsCheckPrewrite</code> / <code>StaticPostGen</code>	Navigation doc officielle + conseil + pre-write + post-check conformité.
<code>WriteLaneGuard</code>	Anti-régression + conformity (symboles requis) pour le write-lane.

5. Routing intelligent des modèles

Module `IntelligentModelRouter` + `TaskClassifier` + `ModelCandidate` + `ModelPerformanceRegistry` + `ModelScoreboard` + `ModelCatalogService` + `DeterministicRouter`.

- **Politiques :** `LOCAL_FIRST`, `FREE_FIRST`, `FREE_UNTIL_EXHAUSTED`, `PASS_FIRST`, `BALANCED`, `QUALITY_FIRST_WITH_BUDGET`, `MANUAL`, `FREE_ONLY`.
- **Classes de tâche :** `MICRO_EDIT`, `DOCUMENTATION`, `UNIT_TEST`, `SMALL_BUG`, `SMALL_FEATURE`, `MEDIUM_FEATURE`, `REFACTOR`, `ARCHITECTURE`.
- **Sélection :** filtre capacités + contexte, applique la politique, gate chaque choix payant via `ProjectBudgetGovernor`. *Déterministe, ne mute jamais le budget.*
- **Output :** modèle choisi + raisons + alternatives + effet budget + confiance.
- **Diagnostic :** `ToolCallClassification` / `ToolCallDiagnostic` (analyse des appels d'outils).

6. Budget & coût

Module	Fonction
<code>ProjectBudgetGovernor</code>	Plafond journalier pays-as-you-go par projet, réservations atomiques (TTL 30 min) empêchant les agents concurrents de dépasser. Modes <code>OFF</code> / <code>ASK_EVERY_TIME</code> / <code>AUTO_WITHIN_PROJECT_BUDGET</code> .
<code>BudgetReservation</code>	Réservation de budget (id, montant, expires).
<code>CostModel</code>	Modèle de coût (SDK_BILLED / LOCALAI / CATALOG_ESTIMATE / PROJECT / UNKNOWN) + qualité.
<code>CostLedger</code>	Registre des coûts encourus.
<code>BudgetStatus</code>	Statut budget.
<code>BudgetSimulator</code>	Simulation de scénarios budget.

Règle d'honnêteté : LocalAI = `EXACT_ZERO` (jamais un prix inventé) ; pay-as-you-go = `EXACT_BILLED` ; sinon ESTIMATED / SUBSCRIPTION_COVERED / UNKNOWN.

7. Télémétrie tokens & coût

Module	Fonction
<code>TokenLedger</code>	Registre d'usage tokens par session (input/output/total), qualité (EXACT/DERIVED/ESTIMATED/UNKNOWN).
<code>WorkspaceRegistry</code>	Association session → workspace (état de la ligne de statut).
<code>UsageObservation</code>	Observation unitaire d'usage (modelId, providerId, tokens).
<code>UsageReport</code>	Consolidation job : tokens total (input+output) + coût + perf (TTFT, tokens/s, VRAM, correctness, stability).
<code>HistoricalUsageImporter</code>	Import de sessions historiques.
<code>numberFormat</code> / <code>tokenEstimate</code>	Formatage + estimation de tokens.
<code>statusBarMode</code>	Quel total afficher : session / workspace / project / off.

8. Artifacts (artefacts)

Objets versionnés avec cycle de vie complet.

- **Types** (26) : `implementation_plan`, `change_contract`, `project_dna`, `workspace_drift`, `architecture`, `dependency_graph`, `project_map`, `code_diff`, `changed_files`, `markdown`, `documentation`, `test_report`, `bug_report`, `audit_report`, `benchmark`, `chart`, `image`, `screenshot`, `terminal_log`, `command_log`, `build_report`, `qa_report`, `security_report`, `performance_report`, `release_report`, `checkpoint`, `ADR`, `reference`.
- **Statuts** (11) : DRAFT, GENERATING, READY_FOR_REVIEW, CHANGES_REQUESTED, APPROVED, APPLYING, VERIFYING, VERIFIED, FAILED, SUPERSEDED, ARCHIVED.
- **Machine à états** : `ArtifactStateMachine` (transitions valides, terminal, canTransition, nextStatuses).

- **Contenu** : index + contenu séparé, SHA256, version auto-incrémentée, comments, pinned, archived, verification+evidence, sourceFiles.
- **Registry** : `ArtifactRegistry` (list/get/readContent).

9. Autonomie (write-lane gated)

L'agent autonome travaille dans un cadre **borné et approuvé**.

1. `/autonomy plan` → `buildAutonomyPlan` (budget minutes par complexité + checkpoints + deadline).
2. `buildWritePlan` + `buildWriteScope` → plan de write-lane : opérations autorisées (create/modify/patch), `maxFiles`, `maxBytesChanged`, `maxPatchLines`, delete/rename bloqués par défaut, `expiresAt`.
3. `guardWritePath` → refuse hors workspace (transversal), fichiers protégés (`.git/config`, `node_modules`), secrets (`.env`, clés).
4. **Approbation explicite** avant toute mutation (jamais d'écriture aveugle).
5. Boucle : activité journalisée + `summary` + handoff.
6. `AutonomyModelSelector` : choisit le modèle selon la complexité.
7. `AutonomyActivityLedger` : suivi minute par minute (détecte le dépassement de budget).

10. Endurance GPU Ladder

Validation de création de projet **obligatoire sur GPU** (pas de fallback CPU). `EnduranceLadder` (machine à états pure, séquence 5→10→20→30→60 sans skip).

- **Gate dur** : le timer ne démarre que si `GPU_OFFLOAD_PROOF = PASS` (GPU détecté + NVIDIA + backend LocalAI GPU + VRAM libre + modèle éligible + sécurité + $\leq 4B$ + FLASH_READY). Sinon `BLOCKED_GPU`.
- **Reset-on-failure** : échec/bug/OOM → temps accumulé perdu, reprise à 00:00 sur le **même** palier (attempt++).
- **Modules** : `EnduranceLedger` (bug + GPU entries), `EnduranceReset` (reset-from-zero + isolation), `ProjectCreationEvidence` (model record + evidence pack), `EnduranceFailover` (failover dédié), `GpuBenchmark` (TTFT/tokens-s/VRAM/stability), `FleetQualifier` (qualification flotte), `EnduranceRouter`, `EnduranceLab` (API produit unique), `GpuQualificationQueue`.
- **Policy** : `requireGpu=true`, `requireFlashReady=true`, `allowCpuFallback=false`. **Jamais** de PASS simulé ni de kill de process.
- **Preuve offload** : delta VRAM (nvidia-smi avant/après) + `/backend/monitor?model=` + smoke inference.
`GpuBenchmark` : 1 warmup + ≥ 3 mesures.

Résultat final : ladder 5/5 passée (5-10-20-30-60 min) → `PROJECT_CREATION_ENDURANCE_READY = PASS`.

11. Guards de sécurité

Guard	Fonction
<code>WorkspaceTrustGuard</code>	Décide si une opération est permise (write/...) selon l'état de confiance du workspace + raison du refus.
<code>WorkspaceTopology</code>	Classifie SINGLE_ROOT/MULTI_ROOT/EMPTY + racine canonique.
<code>AntigravityIDERestartGuard</code>	Valide workspace + autorise redémarrage IDE (jamais Windows).
<code>AntigravityProcessResolver</code>	Résout la fenêtre top-level du process IDE (confidence PROVEN/STRONG/WEAK/AMBIGUOUS/NOT_FOUND).

<code>AutonomyWriteScope</code> / <code>guardWritePath</code>	Write-lane borné : chemins protégés/secrets/bloqués + ops contrôlées.
<code>WriteLaneGuard</code>	StaticPostGen (conformité signatures officielles) + anti-régression (symboles requis).
<code>ProviderPolicyValidator</code>	Valide provider = <code>openai-compatible</code> + base URL loopback + pas de fallback + pas de cloud + router <code>allowCpuFallback=false</code> .
<code>ApprovalBridge</code> / <code>WriteApproval</code>	Pont d'approbation Cline + approbation d'écriture.
<code>PermissionsAdapter</code>	Adaptateur de permissions.
<code>OfficialDocsGuard</code> / <code>ReasoningResponseNormalizer</code>	Gardien doc officielle + normalisation raisonnement.

12. Télémétrie / observabilité / dashboard

- `OutputChannel` (`StructuredLogger`) : logs structurés.
- `ProductionLogger` : format de sortie de prod canonique.
- `DashboardRenderer` : layout fixe (header + k/v + progress + TODO barré + footer).
- `LiveDashboard` : dashboard temps réel (refresh in-place, détecte l'atteinte du goal).
- `TextOutputFormatter` : headers, KV alignés, barres de progress, **TODO barré**.
- `ReasoningResponseNormalizer` , `StreamingBuffer` , `RuntimeEventNormalizer` , `InferenceEvidence` .

13. CLI C++ (menus)

CLI C++/CMake qui pilote Project OS par menus, invoquant la passerelle Node (`bin/project-os-bridge.mjs`).

Fichier	Rôle
<code>cli-cpp/src/pos_json.hpp</code>	Mini-parseur JSON header-only.
<code>cli-cpp/src/pos_model.hpp</code>	Lecture registry/project/goal/todo + helpers + shellQuote.
<code>cli-cpp/src/pos_runner.hpp</code>	Invocation passerelle (<code>_popen</code>) + parsing JSON résultat.
<code>cli-cpp/src/project_os_cli.cpp</code>	Menu interactif (list/create/goal/todo/autonomy/docs/addon/slash/quit).
<code>cli-cpp/tests/test_pos.cpp</code>	Tests JSON/modèle (ctest).

Build : `cmake -S . -B cmake-build -G "MinGW Makefiles"; cmake --build cmake-build` puis `cmake-build\project-os-cli.exe` .

14. Configuration (settings)

Setting	Défaut	Description
<code>providerId</code>	anthropic	Provider ID des sessions.
<code>modelId</code>	claude-sonnet-4-6	Modèle des sessions.

<code>enableTools</code>	true	Outils intégrés (bash, editor, read, search...).
<code>providerType</code>	openai-compatible	Famille provider.
<code>providerBaseUrl</code>	http://localhost:8080/v1	Endpoint OpenAI-compatible/local.
<code>providerModelId</code>	qwen3-4b	Modèle préflight/session.
<code>statusBarMode</code>	project	Quel total tokens afficher (session/workspace/project/off).
<code>routingPolicy</code>	FREE_UNTIL_EXHAUSTED	Politique de routage (8 valeurs).
<code>dailyPaidBudget</code>	0	Plafond pay-as-you-go journalier.
<code>budgetCurrency</code>	USD	Devise du budget.
<code>paidInferenceMode</code>	OFF	Mode inférence payante.
<code>allowLocalAI</code> / <code>allowClineFree</code> / <code>allowClinePass</code> / <code>allowClinePayg</code>	true/true/true/false	Augmente/quels candidats autorisés au routing.
<code>restartAntigravityAfterUpdate</code>	true	Gate de redémarrage IDE après MAJ validée (jamais Windows).
<code>projectsRoot</code>	C:\Users\eiden\Desktop\dev\projects	Racine des workspaces gérés.
<code>activeProjectSlug</code>	""	Slug du projet actif.
<code>defaultAutonomyMinutes</code>	0	Budget minutes /autonomy (0 = auto).
<code>autonomyWriteEnabled</code>	true	Active le write-lane (borné + gated).

15. Tests & baseline

`src/tests/*` : ≥44 fichiers de test, node:test. Baseline gardée : `typecheck 0` · tests **247/247 PASS** · régression 0 · tests C++ **ALL PASS**.

Domaines couverts : slash commands, autonomy, autonomy routing, todo, docs, write-lane, writeLaneGuard, staticPostGen, dashboard, liveDashboard, textOutput, provider preflight/profile, routing budget, artifacts (registry/stateMachine), permissions, approvalBridge, antigravity guards, workspace, streamingBuffer, outputChannel, tokenIntelligence, usageReport, toolCallDiagnostic, projectFactory, managedProjectRegistry.

16. Artefacts livrés (phases 13-23)

`artifacts/` : `antigravity-restarts` , `endurance` (PROJECT_CREATION_ENDURANCE, GPU_LEDGER, BUG_LEDGER, ENDURANCE_GATE_PROOF, FLEET_QUALIFICATION, ENDURANCE_ORCHESTRATOR_REPORT, ENDURANCE_ROUTER_PROOF, ENDURANCE_LAB_REPORT, GPU_CONSUMER_INVENTORY, GPU_CONFIG_TUNING, FLEET_QUALIFICATION), `model-security` , `phase17/18/19/20/21/22/23` , `sfl` (SFL_USAGE_REPORT, AUTONOMY_USAGE_REPORT, dashboard, GOAL_REACHED), `FLASH_FLEET_MANIFEST.json` .

17. Chemins & prérequis

Élément	Chemin
Repo	C:\Users\eiden\Desktop\dev\projet-os
Projets gérés	C:\Users\eiden\Desktop\dev\projects\<slug>
CLI C++	C:\Users\eiden\Desktop\dev\projet-os\cli-cpp\cmake-build\project-os-cli.exe
Passerelle	C:\Users\eiden\Desktop\dev\projet-os\bin\project-os-bridge.mjs
LocalAI	http://127.0.0.1:8080/v1
Toolchain C++	C:\msys64\mingw64\bin

Règles non négociables : jamais de kill de process ni reset Docker/Windows (l'utilisateur contrôle les apps GPU externes) ; jamais de CPU fallback, de PASS simulé ni de qualification inventée ; licence ≤4B, SHA vérifiée, Defender « NO KNOWN MALWARE INDICATORS » ; `trust_remote_code=false` obligatoire.